

Topic 4 Chapters 5 and 6 Worksheet

In the worksheets for this class, you are going to create MySQL statement for each task described below.

You are required to take a screen shot of the SQL statements and the output for each task.

Paste the screenshot of your results under the numbered question.

Save your work and upload the document to the assignment folder by the due date in the Syllabus.

Databases

- New database created during the steps below. (Use the CREATE DATABASE command, followed by USE to switch to that database.)
- Oracle Tutorial database from last week's assignment

Configuration

- The UPDATE and DELETE commands are potentially dangerous, allowing you to make huge database changes with a single command. MySQL helps you prevent this with "Safe Updates". Enable or disable this feature:
SET sql_safe_updates = 1; -- turn on safe updates
SET sql_safe_updates = 0; -- turn off safe updates

1. Create the following table:

```
CREATE TABLE Sinclair (  
  classID INT(10),  
  department VARCHAR(30),  
  deptShort VARCHAR(3),  
  class_number VARCHAR(4),  
  section_number CHAR(3),  
  class_type VARCHAR(10),  
  credit_hours DOUBLE(4, 2),  
  location VARCHAR(15),  
  instructor VARCHAR(20)  
);
```

2. Populate the table with your classes, and then add the following classes:

```
INSERT INTO Sinclair VALUES (001,'REAL ESTATE', 'RES','1101','RA1','REMOTE',3,'REMOTE','Gagnon');  
INSERT INTO Sinclair VALUES (002,'REAL ESTATE', 'RES','1201','RB1','REMOTE',3,'REMOTE','Davis');  
INSERT INTO Sinclair VALUES (003,'REAL ESTATE', 'RES','1301','RA1','REMOTE',3,'REMOTE','Ashton');  
INSERT INTO Sinclair VALUES (004,'CS', 'CIS','2268','RN1','REMOTE',3,'REMOTE','Taylor');  
INSERT INTO Sinclair VALUES (005,'CS', 'CIS','1202','501','ONLINE',3,'ONLINE','Sommer');
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	classID	department	deptShort	class_number	section_number	class_type	credit_hours	location	instructor
▶	1	REAL ESTATE	RES	1101	RA1	REMOTE	3.00	REMOTE	Gagnon
	2	REAL ESTATE	RES	1201	RB1	REMOTE	3.00	REMOTE	Davis
	3	REAL ESTATE	RES	1301	RA1	REMOTE	3.00	REMOTE	Ashton
	4	CS	CIS	2268	RN1	REMOTE	3.00	REMOTE	Taylor
	5	CS	CIS	1202	501	ONLINE	3.00	ONLINE	Sommer

3. Update the table by changing the word "remote" to "virtual".

```
3 • SET sql_safe_updates = 0;
4
5 • UPDATE Sinclair
6 SET class_type = 'virtual'
7 WHERE class_type = 'REMOTE';
8
9 • UPDATE Sinclair
10 SET location = 'virtual'
11 WHERE location = 'REMOTE';
12
13 SELECT *
14 FROM Sinclair;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	classID	department	deptShort	class_number	section_number	class_type	credit_hours	location	instructor
▶	1	REAL ESTATE	RES	1101	RA1	virtual	3.00	virtual	Gagnon
	2	REAL ESTATE	RES	1201	RB1	virtual	3.00	virtual	Davis
	3	REAL ESTATE	RES	1301	RA1	virtual	3.00	virtual	Ashton
	4	CS	CIS	2268	RN1	virtual	3.00	virtual	Taylor
	5	CS	CIS	1202	501	ONLINE	3.00	ONLINE	Sommer

4. Update RES 1301 to be only 1.5 credit hours.

```
17 • UPDATE Sinclair
18 SET credit_hours = 1.5
19 WHERE deptShort = 'RES'
20 AND class_number = 1301;
21
22 • SELECT *
23 FROM Sinclair;
24 -----
```

Result Grid

 Filter Rows:

 Export:

 Wrap Cell Content:

	classID	department	deptShort	class_number	section_number	class_type	credit_hours	location	instructor
▶	1	REAL ESTATE	RES	1101	RA1	virtual	3.00	virtual	Gagnon
	2	REAL ESTATE	RES	1201	RB1	virtual	3.00	virtual	Davis
	3	REAL ESTATE	RES	1301	RA1	virtual	1.50	virtual	Ashton
	4	CS	CIS	2268	RN1	virtual	3.00	virtual	Taylor
	5	CS	CIS	1202	501	ONLINE	3.00	ONLINE	Sommer

5. Create a new table called `Sinclair_copy` from the `Sinclair` table.

```
26 • CREATE TABLE Sinclair_copy AS
27 SELECT *
28 FROM Sinclair;
29
30 • SELECT *
31 FROM Sinclair_copy;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	classID	department	deptShort	class_number	section_number	class_type	credit_hours	location	instructor
▶	1	REAL ESTATE	RES	1101	RA1	virtual	3.00	virtual	Gagnon
	2	REAL ESTATE	RES	1201	RB1	virtual	3.00	virtual	Davis
	3	REAL ESTATE	RES	1301	RA1	virtual	1.50	virtual	Ashton
	4	CS	CIS	2268	RN1	virtual	3.00	virtual	Taylor
	5	CS	CIS	1202	501	ONLINE	3.00	ONLINE	Sommer

For the rest of the assignment you will use the Oracle Tutorial database.

6a. Sum the quantity of items in the inventories

```
6 • SELECT sum(quantity) AS 'total items on hand'
7 FROM inventories;
```

result Grid			Filter Rows:	Export: 	Wrap C
total items on hand					
119512					

6b. Group the quantities by warehouse

```
42 • SELECT warehouse_id, SUM(quantity) AS 'number of items per warehouse'
43 FROM warehouses JOIN inventories USING(warehouse_id)
44 GROUP BY warehouse_id;
45
```

result Grid		 Filter Rows:	Export: 	Wrap Cell Content: 
warehouse_id	number of items per warehouse			
1	5483			
2	28613			
3	7252			
4	14860			
5	12969			
6	20457			
7	9039			
8	13482			
9	7357			

7. Are there any nulls in the manager_id column in the employees table?

Why?

```
16 • SELECT *
17 FROM employees
18 WHERE manager_id is NULL;
```

Result Grid								
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:		
employee_id	first_name	last_name	email	phone	hire_date	manager_id	job_title	
1	Tommy	Bailey	tommy.bailey@example.com	515.123.4567	2016-06-17	NULL	President	
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	

Yes, because he is the president of the company and sits at the very top of the organizational hierarchy. Since there is no higher-ranking employee above him, his manager_id cannot point to anyone and therefore remains NULL.

8. In the products table, what is the most expensive list item, the least expensive item, and what is the average list price?

Put all of the above in one statement.

```
22 • SELECT MAX(list_price) AS 'highest price',
23          MIN(list_price) AS 'lowest price',
24          AVG(list_price) AS 'average list price'
25 FROM products;
```

Result Grid			
Filter Rows:		Export:	
highest price	lowest price	average list price	
8867.99	15.55	903.241389	

9. Take the results from above and group by category name (not id)

```
60 • SELECT
61     category_name,
62     MAX(list_price) AS 'highest price',
63     MIN(list_price) AS 'lowest price',
64     AVG(list_price) AS 'average list price'
65 FROM products JOIN product_categories USING(category_id)
66 GROUP BY category_name;
67
```

Result Grid Filter Rows: Export: Wrap Cell Content:				
	category_name	highest price	lowest price	average list price
▶	CPU	3410.46	554.99	1386.966429
	Video Card	5499.99	739.99	1406.098000
	Mother Board	948.99	279.99	402.293167
	Storage	8867.99	15.55	635.216481

10. Use the statement above and add an average column for all items

Hint: see page 187 or 189 (Depends on your edition of the textbook. Look for the OVER() instruction and partitioning.)

```
70 • SELECT
71     pc.category_name,
72     MAX(p.list_price) AS 'highest price',
73     MIN(p.list_price) AS 'lowest price',
74     AVG(p.list_price) AS 'average list price by category',
75     AVG(ANY_VALUE(p.list_price)) OVER() AS 'Average of all items'
76 FROM products p
77 JOIN product_categories pc USING(category_id)
78 GROUP BY pc.category_name;
```

Result Grid Filter Rows: Export: Wrap Cell Content:					
	category_name	highest price	lowest price	average list price by category	Average of all items
▶	CPU	3410.46	554.99	1386.966429	1118.990000
	Video Card	5499.99	739.99	1406.098000	1118.990000
	Mother Board	948.99	279.99	402.293167	1118.990000
	Storage	8867.99	15.55	635.216481	1118.990000

```

78 -- 903.24 = the average list price of all products.
79 -- The window function calculates the average before the GROUP BY is applied.
80 • SELECT DISTINCT
81     pc.category_name,
82     MAX(p.list_price) OVER category_window AS 'highest price',
83     MIN(p.list_price) OVER category_window AS 'lowest price',
84     AVG(p.list_price) OVER category_window AS 'average list price',
85     AVG(p.list_price) OVER() AS 'Average price of all products'
86 FROM products p JOIN product_categories pc USING(category_id)
87 WINDOW category_window AS (PARTITION BY category_name ORDER BY category_id);
88

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	category_name	highest price	lowest price	average list price	Average price of all products
▶	CPU	3410.46	554.99	1386.966429	903.241389
	Mother Board	948.99	279.99	402.293167	903.241389
	Storage	8867.99	15.55	635.216481	903.241389
	Video Card	5499.99	739.99	1406.098000	903.241389

```

89 -- 957.64 = the average of the four category averages.
90 -- Each category has equal weight, regardless of the number of products in the category.
91 • SELECT
92     c.category_name,
93     MAX(p.list_price) AS 'highest price',
94     MIN(p.list_price) AS 'lowest price',
95     AVG(p.list_price) AS 'average list price',
96     AVG(AVG(p.list_price)) OVER () AS 'average of category averages'
97 FROM products p JOIN product_categories c USING (category_id)
98 GROUP BY c.category_name;

```

Result Grid   Filter Rows: Export:  Wrap Cell Content: 

	category_name	highest price	lowest price	average list price	average of category averages
▶	CPU	3410.46	554.99	1386.966429	957.6435192500
	Video Card	5499.99	739.99	1406.098000	957.6435192500
	Mother Board	948.99	279.99	402.293167	957.6435192500
	Storage	8867.99	15.55	635.216481	957.6435192500